

POSTGRESQL ESSENTIALS FOR ODOO DATABASES

PostgreSQL for Odoo

SQL, roles, indexes, and backup basics

Merit Advisory

Odoo Full-Stack Developer Program

Beginner-friendly lecture materials

July 2026

July 2026



Agenda

- 1 Section 1: Install PostgreSQL on Ubuntu to List Databases and Tables — 7 topics
- 2 Section 2: Data Types Overview to Update Data — 7 topics
- 3 Section 3: Delete Data to Group Rows with GROUP BY — 7 topics
- 4 Section 4: Filter Groups with HAVING to Understand the pg_hba.conf Idea — 7 topics
- 5 Section 5: Understand the postgresql.conf Idea to Keep a Backup Habit for Odoo — 5 topics

1 Section 1

Install PostgreSQL on Ubuntu to List Databases and Tables

1.1 Install PostgreSQL on Ubuntu

- Use apt for a standard Ubuntu install.
- Install contrib tools for common extensions.

```
user@odoo-lab:~/project

trainee@odoo-lab:~$ sudo apt install -y postgresql postgresql-contrib
Setting up postgresql (16.x) ...
trainee@odoo-lab:~$ psql --version
psql (PostgreSQL) 16.3
```

1.2 Start and Check the PostgreSQL Service

- PostgreSQL usually runs as a systemd service.
- Enable the service to start after reboot.

```
user@odoo-lab:~/project

trainee@odoo-lab:~$ sudo systemctl enable --now postgresql
trainee@odoo-lab:~$ systemctl is-active postgresql
active
```

1.3 Connect with psql

- psql is the main PostgreSQL command-line client.
- Local admin access often uses the postgres OS user.

```
psql -d training

training=# SELECT id, name FROM partners;
 id | name 
----+-----
  1 | Ada 
(1 row)
```

1.4 Create a PostgreSQL Role

- Roles are database identities.
- LOGIN lets a role connect like a user.

```
psql -d training

training=# SELECT id, name FROM partners;
 id | name 
----+-----
  1 | Ada 
(1 row)
```

1.5 Create a Database

- A database stores schemas, tables, and data.
- Set the owner when the database is created.

```
psql -d training

training=# SELECT id, name FROM partners;
 id | name 
-----+-----
  1 | Ada 
(1 row)
```


1.6 Grant Database Permissions

- Grants control what a role can do.
- Database grants are not the same as table grants.

```
psql -d training

training=# SELECT id, name FROM partners;
 id | name 
-----+-----
  1 | Ada 
(1 row)
```

1.7 List Databases and Tables

- psql meta commands start with a backslash.
- \l lists databases.

```
psql -d training

training=# SELECT id, name FROM partners;
 id | name 
-----+-----
  1 | Ada 
(1 row)
```

2 Section 2

Data Types Overview to Update Data

2.1 Data Types Overview

- Choose types that match the meaning of data.
- Use text types for names and descriptions.

```
psql -d training

training=# SELECT id, name FROM partners;
 id | name 
-----+-----
  1 | Ada 
(1 row)
```

2.2 Create a Table

- Tables define columns and constraints.

Simple table

```
CREATE TABLE customers (  
    id integer,  
    name text,  
# ...
```

- Use consistent names for columns.

Table with defaults

```
CREATE TABLE notes (  
    id integer,  
    body text,  
# ...
```

2.3 Use a Primary Key

- A primary key uniquely identifies each row.

Identity primary key

```
CREATE TABLE products (  
    id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    name text NOT NULL  
# ...
```

- It cannot be null.

Natural primary key

```
CREATE TABLE countries (  
    code char(2) PRIMARY KEY,  
    name text NOT NULL  
# ...
```

2.4 Use a Foreign Key

- A foreign key links one table to another.

Customer orders

```
CREATE TABLE orders (  
    id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    customer_id bigint REFERENCES customers(id)  
# ...
```

- It protects data from broken references.

Required relationship

```
product_id bigint NOT NULL REFERENCES products(id)
```

2.5 Insert Data

- INSERT adds new rows.

Insert one row

```
INSERT INTO customers (name, email)
VALUES ('Ada', 'ada@example.com');
```

- List columns to avoid order mistakes.

Insert and return id

```
INSERT INTO products (name)
VALUES ('Keyboard')
RETURNING id, name;
```


2.6 Read Data with SELECT

- SELECT reads rows from tables.

Select columns

```
SELECT id, name, email  
FROM customers;
```

- Choose only the columns you need.

Sort results

```
SELECT name, price  
FROM products  
ORDER BY name;
```

2.7 Update Data

- UPDATE changes existing rows.

Update one customer

```
UPDATE customers  
SET email = 'ada.new@example.com'  
WHERE id = 1;
```

- Always check the WHERE clause.

Update and review

```
UPDATE products  
SET price = 19.99  
WHERE name = 'Keyboard'  
# ...
```

3 Section 3

Delete Data to Group Rows with GROUP BY

3.1 Delete Data

- DELETE removes rows from a table.

Delete one row

```
DELETE FROM customers  
WHERE id = 10;
```

- Use WHERE to target specific rows.

Preview then delete

```
SELECT * FROM products WHERE name = 'Old Item';  
DELETE FROM products WHERE name = 'Old Item';
```

3.2 Filter Rows with WHERE

- WHERE limits which rows are returned.

One condition

```
SELECT * FROM customers  
WHERE email IS NOT NULL;
```

- Combine filters with AND and OR.

Multiple conditions

```
SELECT * FROM products  
WHERE price > 10 AND is_active = true;
```

3.3 Search Text with LIKE and ILIKE

- LIKE matches text patterns.

Case-sensitive search

```
SELECT name FROM customers  
WHERE name LIKE 'A%';
```

- ILIKE ignores letter case.

Case-insensitive search

```
SELECT name FROM customers  
WHERE name ILIKE '%smith%';
```

3.4 Filter with IN and BETWEEN

- IN checks a value against a list.

Use IN

```
SELECT * FROM orders  
WHERE status IN ('draft', 'sent');
```

- BETWEEN checks an inclusive range.

Use BETWEEN

```
SELECT * FROM invoices  
WHERE amount_total BETWEEN 100 AND 500;
```

3.5 Combine Rows with INNER JOIN

- INNER JOIN returns matching rows only.

Orders with customers

```
SELECT o.id, c.name  
FROM orders o  
INNER JOIN customers c ON c.id = o.customer_id;
```

- Join conditions usually compare keys.

Lines with products

```
SELECT l.id, p.name  
FROM order_lines l  
JOIN products p ON p.id = l.product_id;
```


3.6 Keep Rows with LEFT JOIN

- LEFT JOIN keeps all rows from the left table.

Customers with optional orders

```
SELECT c.name, o.id AS order_id  
FROM customers c  
LEFT JOIN orders o ON o.customer_id = c.id;
```

- Missing matches appear as null values.

Products with optional categories

```
SELECT p.name, c.name AS category  
FROM products p  
LEFT JOIN categories c ON c.id = p.category_id;
```

3.7 Group Rows with GROUP BY

- GROUP BY collects rows into groups.

Orders by status

```
SELECT status, COUNT(*)  
FROM orders  
GROUP BY status;
```

- Use aggregates to summarize each group.

Sales by customer

```
SELECT customer_id, SUM(amount_total)  
FROM invoices  
GROUP BY customer_id;
```

4

Section 4

Filter Groups with HAVING to Understand the pg_hba.conf Idea

4.1 Filter Groups with HAVING

- HAVING filters grouped results.

Customers with many orders

```
SELECT customer_id, COUNT(*)  
FROM orders  
GROUP BY customer_id  
# ...
```

- WHERE filters rows before grouping.

Large sales groups

```
SELECT customer_id, SUM(amount_total)  
FROM invoices  
GROUP BY customer_id  
# ...
```

4.2 Use COUNT, SUM, and AVG

- COUNT counts rows or values.

Count rows

```
SELECT COUNT(*) AS order_count  
FROM orders;
```

- SUM adds numeric values.

Sales summary

```
SELECT SUM(amount_total), AVG(amount_total)  
FROM invoices;
```

4.3 Create an Index

- Indexes help PostgreSQL find rows faster.

Index a search column

```
CREATE INDEX idx_customers_email ON customers (email);
```

- Index columns used often in filters or joins.

Index a foreign key

```
CREATE INDEX idx_orders_customer_id ON orders (customer_id);
```

4.4 Read Plans with EXPLAIN ANALYZE

- EXPLAIN shows how PostgreSQL plans a query.

Inspect a select

```
EXPLAIN ANALYZE
SELECT * FROM customers WHERE email = 'ada@example.com';
```

- ANALYZE runs the query and measures time.

Inspect a join

```
EXPLAIN ANALYZE
SELECT o.id, c.name
FROM orders o
# ...
```

4.5 Back Up with pg_dump

- pg_dump creates a logical backup.

Custom format backup

```
pg_dump -h 127.0.0.1 -U odoo17 -Fc -f odoo17.dump odoo17_prod
```

- Custom format is flexible for restore.

Plain SQL backup

```
pg_dump -U odoo17 -f training_db.sql training_db
```


4.6 Restore with pg_restore

- pg_restore reads custom-format dumps.

Create target database

```
createdb -h 127.0.0.1 -U odoo17 odoo17_restore
```

- Restore into the correct target database.

Restore dump

```
pg_restore -h 127.0.0.1 -U odoo17 -d odoo17_restore odoo17.dump
```

4.7 Understand the pg_hba.conf Idea

- pg_hba.conf controls client authentication.

Local password rule

```
host    all         odoo17    127.0.0.1/32    scram-sha-256
```

- Rules are checked from top to bottom.

Reload configuration

```
sudo systemctl reload postgresql
```

5 Section 5

Understand the postgresql.conf Idea to Keep a Backup Habit for Odoo

5.1 Understand the postgresql.conf Idea

- postgresql.conf controls server behavior.

Listen locally

```
listen_addresses = 'localhost'  
port = 5432
```

- Common settings include port and listen address.

Show config file

```
SHOW config_file;  
SHOW listen_addresses;
```

5.2 See Active Connections

- pg_stat_activity shows current sessions.

View active sessions

```
SELECT pid, username, datname, state  
FROM pg_stat_activity;
```

- It helps identify users, databases, and queries.

Find Odoo sessions

```
SELECT pid, state, query  
FROM pg_stat_activity  
WHERE username = 'odoo17';
```

5.3 Cancel or Terminate a Query

- Cancel asks a query to stop cleanly.

Cancel a query

```
SELECT pg_cancel_backend(12345);
```

- Terminate closes the whole session.

Terminate a session

```
SELECT pg_terminate_backend(12345);
```

5.4 Use a Dedicated Odoo Database User

- Do not run Odoo as the postgres superuser.

Development role

```
CREATE ROLE odoo17_dev LOGIN CREATEDB PASSWORD 'change-me';
```

- Use one clear role for each Odoo environment.

Connection options

```
db_user = odoo17_dev  
db_password = change-me  
db_host = 127.0.0.1
```

5.5 Keep a Backup Habit for Odoo

- Back up before upgrades and migrations.

Database backup

```
pg_dump -U odoo17 -Fc -f before_upgrade.dump odoo17_prod
```

- Keep database and filestore backups together.

Filestore backup

```
tar -czf filestore_before_upgrade.tgz ~/.local/share/Odoo/filestore/odoo17_prod
```


NEXT STEP

Try the examples on your machine

Then open the next module deck

Merit Advisory
03 · Database

